PRA-MLX: Query-Addressed Memory beside Prompt and Rotating KV Caches

Jorge Simão
EInnovator R&D

August 2026

Abstract

Apple Silicon changes the physical cost model of long-context inference: weights, cache arrays, and host code share unified memory. It does not remove the logical question of which retained evidence should be active. We implement an MLX adapter boundary for Progressive Retrieval Attention (PRA), measure selected-text coexistence with the MLX-LM nearest-prefix cache, and test whether a rotating sequential cache can serve as a simpler archive. On an Apple M5 with the 4-bit Qwen3-0.6B checkpoint, selected and full context recover a fixed answer in 5/5 server requests; selected context uses 365 rather than 1,577 prompt tokens. Warm requests reuse 364 and 1,576 cached tokens, respectively, while the server prompt cache grows to 0.28 GB across five distinct sequences. In a separate five-seed Python-API experiment, full sequential KV recovers the answer in 5/5 cases with 112 MB of cache. Rotating KV at 64–512 tokens recovers 0/5. Reintroducing the selected archive record reaches 2/5 only at 64 tokens and 0/5 at larger capacities. We then implement native selected K/V at every layer. Across five seeds it is bit-exact to ordinary split-cache execution, recovers 5/5 answers, and remains 5/5 when local sequential K/V rotates at 64 tokens. Thus native PRA restores archived evidence without making it visible prompt text. A follow-up profile sweep finds that the last 14 of 28 layers retain 5/5 recovery with 7.71 rather than 15.42 MiB active K/V. Persisted K/V reloads in 7.2 ms with zero logit error, and shared-memory throughput rises from 6.04 to 16.01 requests/s between concurrency one and eight. On 40 QASPER and 40 HotpotQA natural-text transport controls per condition, ordinary and native memory rank the answer correctly in every case; disabled and shuffled memory remain near chance. A larger Qwen3-1.7B natural-QA study then uses the datasets' original questions and answers. Full-precision native K/V exactly matches ordinary split-cache token F1 and answer log-probability on QASPER, HotpotQA, and 2WikiMultiHopQA. Shuffled and absent memory degrade both metrics on QASPER and HotpotQA; on 2Wiki the gold-answer log-probability degrades while free-generation F1 is noisy. Per-head int8 residency halves selected-K/V bytes and preserves this small cohort's quality, with request-local dequantization before attention. An independent M4 Pro replication adds Qwen3-4B and preserves ordinary/native aggregate F1 exactly in all six model–dataset cohorts. At 4B, native completion takes 44.5–47.6% of ordinary split-cache time; shuffled-memory F1 falls to 0.011–0.028. Finally, an SDK hybrid router selects four documents or paper paragraphs before native materialization. Native and ordinary consumption agree exactly on all 60 paired generations. Gold evidence recall at four is 0.348 on QASPER, 0.725 on HotpotQA, and 0.775 on 2Wiki. Routed memory outperforms shuffled memory on answer likelihood in all three datasets, but remains below oracle evidence; discovery, rather than MLX transport, is now the measured bottleneck. On the same frozen routed selections, a matched E0/E2 economic benchmark preserves all 840 output pairs across cold, warm, multi-query, and concurrency-eight schedules and removes 90.6–93.0% of visible prompt tokens. Native ingestion is cheaper than ordinary source-cache construction, but the unfused selected-cache path yields macro E2/E0 cost ratios of 1.130, 1.155, 1.148, and 1.132 across those schedules. An expanded 149-question confirmation preserves 2,086/2,086 pairs with ratios of 1.043, 1.139, 1.133, and 1.072. A 1,020-request pressure sweep then holds QA quality constant while varying an eight-resource session's compact-K/V budget. Budgets below eight reload every repeated access; fitting all eight resources eliminates reloads and halves mean resolution time at a 151–168 MiB residency cost. An expanded live lifecycle study adds layer-segmented mmap WARM storage: WARM is 80/80 exact and restart recovery succeeds, while int8 COLD is exact in only 13/80 sequences and remains an explicit, unpromoted option. Five-example Llama and Gemma replications also preserve every lossless WARM output, while their small int8 results remain calibration observations rather than defaults. Event-loop promotion hides the WARM stall with sufficient lead, and the native streaming gateway measures queueing through concurrency eight with cancellation-safe cleanup. A 1,125-request long-session extension confirms an eight-resource working-set threshold, while selective int8 remains below the lossless gate. An additional 1,500-request Qwen3-4B M4 Pro sweep reproduces that threshold: quality is budget-invariant, fitting all eight resources eliminates reloads, and mean resolution time falls by about 68%.

1 Qualification Snapshot

The engine-specific question is whether selected native K/V can improve the quality–memory–latency frontier in Apple unified memory beyond selected-text E0. The live native executor is validated at E2 and preserves 2,086/2,086 matched E0 outputs while removing 90.4–93.0% of visible prompt tokens. The current unfused path remains slower: macro E2/E0 cost ratios are 1.043 cold, 1.139 warm, 1.133 multi-query, and 1.072 concurrent. E0 is therefore the default when visible selected text is acceptable; E2 is a functional native mechanism whose memory and non-disclosure properties require workload-specific valuation.

Integration level	E2 validated in-process; no E3 scheduler claim
Model/hardware	Qwen, Llama, and Gemma-family checks on Apple M5; Qwen3-1.7B/4B replication on M4 Pro
Workload	QASPER, HotpotQA, and 2WikiMultiHopQA; frozen selections
Quality	2,086/2,086 matched E0/E2 pairs; six additional oracle-transport cohorts preserve aggregate F1 exactly
Context reduction	90.4–93.0% fewer visible prompt tokens
TTFT/ITL tails	Online gateway measured separately; common matched tails `NOT_MEASURED`
Throughput	Concurrent E2 inverse-cost ratio 1.072, or 93.3% of E0 rate
Peak memory	Unified-memory components and controlled pressure measured; full live frontier open
Reuse	HOT/WARM lifecycle and restart recovery measured
Main limitation	Install segmented one-softmax attention in live decode and rerun pressure

PRA primer. PRA gives each durable semantic resource stable identity, version, authorization, and selectable intervals. E0 renders one frozen selection as visible text. E1 preserves logical identity while executing an ordinary model path. E2 consumes the identical selection as detached native K/V with correct positions, masks, cache topology, one normalization, cleanup, and isolation. E3 adds a scheduler that owns placement, promotion, eviction, sharing, and batching. MLX prompt or rotating caches retain sequential local state; PRA holds independently selected archive state.

Deployment recommendation. **Best validated deployment:** E0 selected text for latency-sensitive serving; E2 for bounded native residency, non-disclosure, or persistent resource reuse after profiling. **Use when:** a selected archive should remain outside the visible prompt. **Do not use when:** a cold one-shot request is expected to outperform E0 without reuse. **Main remaining gate:** live fused attention plus a joint local-window/PRA-budget/consumer-layer frontier.

2 Introduction

MLX provides an efficient array framework and model runtime for Apple Silicon [1]. MLX-LM adds explicit prompt caches, rotating K/V, optional cache quantization, batching, and an OpenAI-like server. These tools make sequential context reuse practical in unified memory. PRA addresses a different object:

$$\text{large retained memory} \rightarrow \text{query-selected region} \rightarrow \text{active native detail.}$$

Unified memory may reduce copy overhead, but it does not make every retained token free to index, keep, or attend to.

This paper studies six questions. First, can selected context coexist with MLX's exact prompt cache? Second, can a bounded rotating cache plus a selected archive record recover old evidence? Third, what software boundary is needed before an MLX adapter may claim native PRA K/V rather than selected visible text? Fourth, how much native K/V can consumer-layer profiles remove without losing the controlled answer? Fifth, do persistence, request sharing, and natural-text causal controls, original benchmark questions, and quantized residency preserve the mechanism? Sixth, when the runtime itself selects evidence, can we separate routing error from native-K/V consumption error?

3 Cache Taxonomy

We keep three mechanisms distinct:

Prompt cache
: exact reusable prompt-prefix K/V, retained by MLX-LM.

Rotating cache
: bounded recent sequential K/V, with old positions evicted as the prompt advances.

PRA memory
: query-addressed non-prefix resources whose address state, native detail, and active selection have separate lifecycles.

Prompt-cache bytes are physical retained state. PRA backing may be much larger than active detail. Rotating capacity bounds direct context but does not create an address for evicted evidence.

4 Implementation

The engine-neutral namespace in `src/pra_hf/engine_memory.py` identifies a block by tenant/session scope, resource/version, token interval, consumer layer, immutable model revision, dtype/layout, materialization profile, and position policy. Its residency state machine records indexed-only, off-device, prefetching, resident, pinned, evictable, and invalid states. In MLX, the tier name denotes array residency or backing policy within unified memory; we do not mislabel it as CUDA host-to-device transfer.

`src/pra_mlx/adapter.py` exposes the HTTP server as E0. It advertises automatic prefix caching and text fallback, but no native K/V. An in-process `MLXNativeExecutor` is required before the adapter advertises E2, selected interval materialization, logical resource deltas, or explicit prompt cache handles. `src/pra_mlx/native.py` now provides that executor.

The design choice is important. Precomputing a prompt cache from selected text still makes that text a sequential prefix. A native executor must instead preserve the selected region's identity and positional policy while combining its K/V with local K/V under correct model attention.

The implementation encodes immutable resource text once and retains post-RoPE K/V in $[B, H_{kv}, T, D]$ form for every layer. A request-local cache returns *selected memory plus local sequential K/V* to each attention module and constructs a mask in which every query sees selected memory while local keys remain causal. Query positions advance from the source extent; selected text never appears in the visible prompt. The residency manager deduplicates materialization, pins blocks for request lifetime, and evicts only unpinned detail under a bounded byte budget.

For compact residency, each K and V head receives a symmetric int8 scale over its token and head-width axes. Compact arrays remain the retained object; selection dequantizes them to the model-native dtype before constructing the request-local attention cache. This version measures storage economy, not an int8 attention kernel: active attention still consumes full-precision K/V.

Consumer-layer profiles preserve the source-relative query position even on layers that omit selected detail. Such a layer receives only request-local K/V, but its RoPE offset still advances from the source extent; otherwise a profile sweep would confound memory removal with a positional-frame change. Persisted arrays carry a strict sidecar fingerprint over model and tokenizer revisions, dtype, position policy, consumer profile, and resource version. A mismatch is rejected before arrays enter attention.

5 Environment

The host was an Apple M5 MacBook Pro with 10 CPU and 10 GPU cores, 16 GB unified memory, Metal 4, and macOS 26.4.1. The environment used Python 3.12.7, MLX 0.32.0, and mlx-lm 0.31.3. The model was `mlx-community/Qwen3-0.6B-4bit` at revision `73e3e38d981303bc594367cd910ea6eb48349da8`. The HTTP server retained up to 16 prompt-cache entries. It emits an explicit warning that its basic security checks are not production-grade; we treat it as an experimental deployment baseline.

The end-task study uses `mlx-community/Qwen3-1.7B-4bit`, MLX-LM 0.31.3, five seeds, and four held-out examples per seed and dataset. Evidence is oracle-scoped from dataset annotations and truncated to 384 tokens; routing quality is deliberately outside the first engine-transport experiment. A replication uses an M4 Pro with 20 GPU cores and 48 GiB unified memory, MLX 0.32.0, and MLX-LM 0.31.3. It repeats the same five seeds for Qwen3-1.7B and Qwen3-4B. Thus hardware and model scale change while dataset selection and execution conditions remain fixed. A second matched-budget cohort retains every HotpotQA and 2Wiki candidate document and every QASPER paper paragraph, then routes four candidates before native materialization.

Table 1: MLX-LM server smoke. TTFT is milliseconds; warm cached is the mean over repeats two through five.

Condition	Exact	Prompt	Cold TTFT	Warm TTFT	Warm cached
None	0%	24	523.6	505.2	23
Prefix	0%	333	446.4	355.2	332
Selected	100%	56	415.9	355.0	55
Prefix + selected	100%	365	432.0	352.3	364
Full context	100%	1577	565.4	351.8	1576

Table 2: Five-seed controlled rotating-cache experiment. Cache MB is the sum of layer-cache `nbytes`; latency is the mean completion time.

Condition	K/V tokens	Exact	Cache MB	Latency ms
Full sequential K/V	full	100%	112.0	468.9
Rotating only	64	0%	7.0	706.6
Rotating only	128	0%	14.0	730.2
Rotating only	256	0%	28.0	722.1
Rotating only	512	0%	56.0	721.2
Rotating + selected archive	64	40%	7.0	704.7
Rotating + selected archive	128	0%	14.0	736.2
Rotating + selected archive	256	0%	28.0	713.1
Rotating + selected archive	512	0%	56.0	732.2

6 Server Prompt-Cache Experiment

The shared fixed task contains a stable 309-token prefix, one authoritative codeword record, twelve distractor records, and a short query. Five conditions separate no memory, prefix only, selected memory only, prefix plus selected memory, and full context. Each is repeated five times. The server reports cached tokens in the streaming usage object, and SSE events determine TTFT.

Table 1 shows correct controls: selected and full conditions are 5/5, while conditions without evidence are 0/5. Prefix plus selected memory uses 365 tokens and full context uses 1,577, again a 76.9% visible reduction. Warm selected requests reuse 364 tokens; warm full requests reuse 1,576. The server's nearest-prefix cache therefore composes cleanly with the selected-text baseline.

Warm TTFT is nearly flat, 352.3 versus 351.8 ms, because exact prompt caches remove most repeated prefill work in both conditions. This is a useful negative latency result: after a perfect repeat cache hit, reducing visible input does not improve warm TTFT on this setup. Physical retention still differs. The log shows five distinct prompt-cache sequences occupying about 0.28 GB at the end of the matrix.

7 Rotating Cache and Selected Archive

The Python-API control uses five deterministic filler permutations. One authoritative four-digit fact appears near the beginning of a roughly thousand-token prompt. We compare full sequential K/V with rotating capacities 64, 128, 256, and 512. For each rotating capacity, a second condition reintroduces the authoritative record immediately before the final question. The answer metric accepts harmless formatting but requires the correct digits.

Full sequential K/V is 5/5 with 112 MB of cache. Rotating-only is 0/5 at every capacity while reducing layer-cache storage from 112 MB to 7, 14, 28, and 56 MB. Selected archive text reaches 2/5 at 64 and 0/5 at larger capacities. Mean completion latency is 469 ms for full K/V and approximately 705–736 ms for bounded conditions. The generated outputs show chat-format and answer-path instability, not merely loss of the early fact.

This non-monotonic result should not be read as evidence that smaller cache is better. It indicates that the direct rotating-cache intervention changes more than evidence availability for this model and prompt. The selected-text fallback does not restore stable computation. A native PRA executor must retain local sequential semantics while adding selected memory separately; the next section evaluates that mechanism directly.

Peak process memory remains about 1.01–1.07 GB across these conditions despite the 7–112 MB cache range. Model and runtime allocations dominate on this small checkpoint, so layer-cache bytes are the interpretable economy metric. Larger models and memory pressure are required before process-level savings claims.

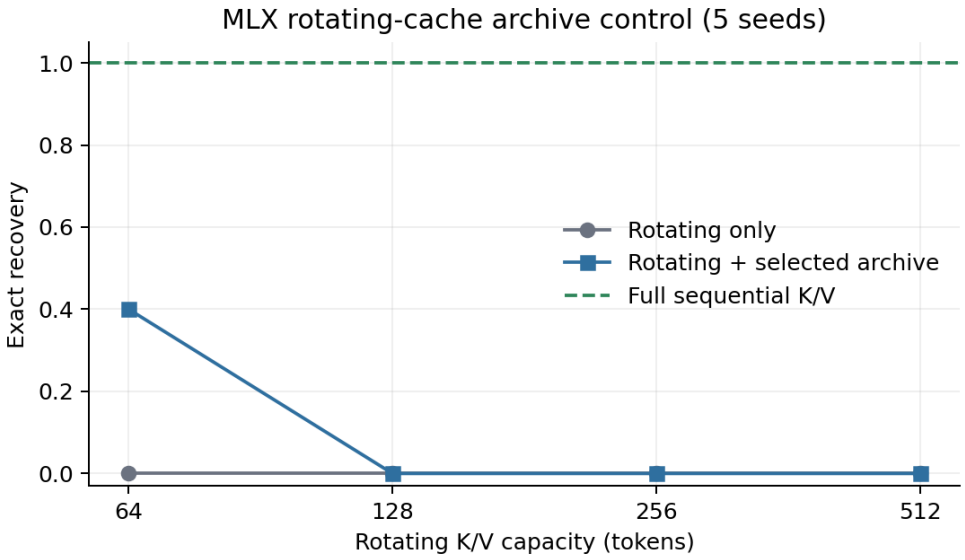

Figure 1: A bounded rotating cache is not a reliable archive in this control. Selected late text recovers two of five cases only at the smallest capacity.

Table 3: Five-seed native selected-K/V control. Latency excludes the separately reported one-time native encoding cost. Native MB is all-layer selected K/V.

Condition	Exact	Latency ms	Native MB	Max error
Ordinary split cache	100%	187.0	0.0	0
Native selected K/V	100%	166.3	15.4	0
Native + rotating local 64	100%	215.5	15.4	0

8 Native Selected-K/V Experiment

The source contains 141 tokens. Ordinary split-cache and native selected-K/V produce identical logits, identical argmax tokens, and 5/5 exact recovery. Native selected execution averages 166.3 ms after a separately measured 55.4 ms one-time encoding step and retains 15.4 MB of all-layer K/V. Most importantly, replacing the local sequential cache by a 64-token rotating cache still recovers 5/5. The selected archive is therefore independent of local rotation rather than late visible text competing inside that bounded cache.

Table 4 repeats the mechanism on Llama 3.2 1B and Gemma 3 1B. Llama achieves 5/5 native and rotating-local recovery with exact split-cache logits while retaining 4.34 MB. Gemma also has zero logit error and matching argmax decisions, but both its ordinary and native conditions are 0/5 on this particular answer-format task. Gemma therefore establishes mechanism parity, not answer recovery. Together the runs cover three attention topologies and tokenizers; they are not yet natural-workload results.

9 Profiles, Persistence, and Concurrency

Table 5 exposes a sharp controlled boundary. Full native memory and the last-half profile both recover 5/5 answers. Last quarter, last four, and disabled profiles recover 0/5. Last half reduces active selected K/V from 15.42 to 7.71 MiB. Its nonzero logit difference is expected because fourteen layers no longer see the archive; exact recovery, not bit parity, is the profile criterion. This is a candidate economy profile for this fixture, not a production profile calibrated from five examples.

Persisting and reloading native arrays yields zero maximum logit error. Mean save and load times are 28.7 and 7.2 ms. Separate request-local caches share one immutable memory object: at concurrency eight this avoids 108.0 MiB of duplicate selected K/V. Aggregate throughput increases through concurrency four and then plateaus, while mean per-request latency rises from 164.9 to 383.9 ms. These are in-process controlled measurements, not queued

Table 4: Cross-model native selected-K/V replication. Native and rotating columns report exact recovery over five seeds; error is zero for every row.

Model	Seeds	Native exact	Rotating exact	Native MB
Qwen3 0.6B 4-bit	5	100%	100%	15.42
Llama 3.2 1B 4-bit	5	100%	100%	4.34
Gemma 3 1B 4-bit	5	0%	0%	3.58

Table 5: Five-seed consumer-layer sweep. Active MB counts selected native K/V, not model weights or request-local K/V.

Consumer profile	Layers	Exact	Active MB	Max error
all	28	100%	15.42	0
disabled	0	0%	0.00	6.188
last_4	4	0%	2.20	4.844
last_half	14	100%	7.71	4.031
last_quarter	7	0%	3.86	4.188

Table 6: Five-seed persistence and concurrent request sharing. Persist load is the mean for a 15.43 MiB selected-K/V sidecar.

Concurrency	Exact	Requests/s	Mean latency ms	Shared MB saved	Persist load ms
1	100%	6.04	164.9	0.0	7.2
2	100%	10.80	180.1	15.4	7.2
4	100%	16.98	222.4	46.3	7.2
8	100%	16.01	383.9	108.0	7.2

server tail latency.

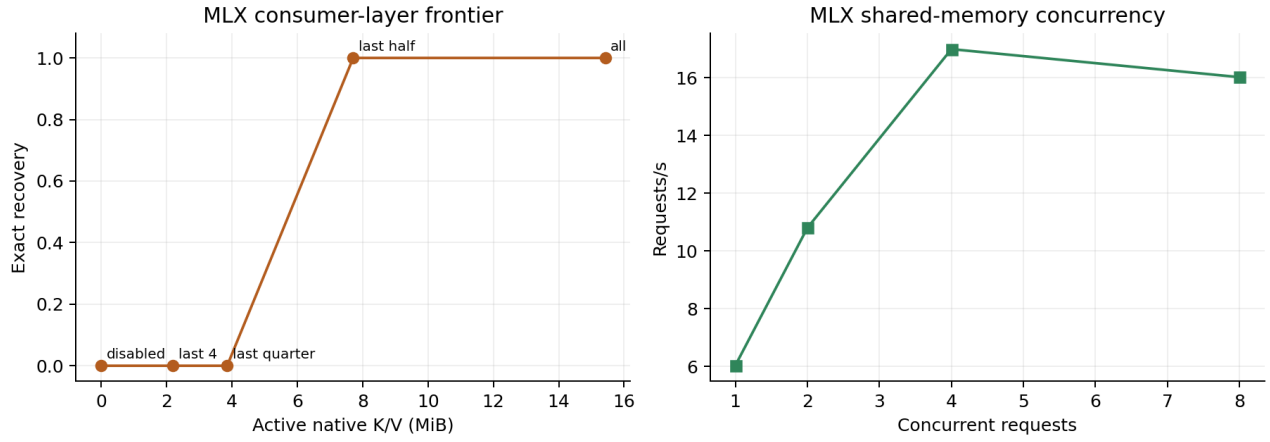

Figure 2: Left: the controlled consumer-layer quality–memory boundary. Right: shared immutable K/V raises throughput until the small Apple host saturates.

10 Natural-Text Causal Controls

The QASPER and HotpotQA builders draw natural paper or article text and place a balanced `AnswerCode yes/no` anchor among 63 source units. They test whether an answer-bearing natural-text resource survives the native transport; they do not claim unrestricted question answering. Each dataset contributes eight examples under each of five seeds, for 40 examples per condition. We rank the next-token logits for `yes` and `no`; free-form generation is

retained only as a diagnostic because this small completion model often ignores the requested answer format.

Table 7: Natural-text transport controls. Margin is the mean gold-minus-other answer logit.

Dataset	Condition	Samples	Ranked exact	Margin
qasper	native_all	40	100%	2.953
qasper	native_disabled	40	50%	0.014
qasper	native_last_half	40	100%	1.009
qasper	native_shuffled	40	50%	-0.108
qasper	ordinary_split	40	100%	2.953
hotpotqa	native_all	40	100%	2.885
hotpotqa	native_disabled	40	52%	-0.002
hotpotqa	native_last_half	40	100%	1.113
hotpotqa	native_shuffled	40	50%	0.166
hotpotqa	ordinary_split	40	100%	2.885

Ordinary split-cache and all-layer native execution agree on all 80 examples and have identical mean margins within each dataset. Last-half execution also ranks all 80 correctly while reducing the margin. Disabled memory is 50.0% on QASPER and 52.5% on HotpotQA; shuffled memory is 50.0% on both. The causal controls therefore rule out query-only label bias as the source of native success for this transport task.

11 Original-Answer QA and Quantized Residency

The next experiment removes the inserted answer code. It evaluates original QASPER, HotpotQA, and 2WikiMultiHopQA questions and short gold answers. For each example, annotated evidence is placed first and natural distractor or abstract text fills the 384-token source budget. This is an oracle-evidence materialization study: it asks whether the engine transports useful native K/V, not whether PRA routing found the evidence.

Table 8: Natural-QA generation with Qwen3-1.7B-4bit over five seeds. Strict EM is zero because the model often continues after the answer; token F1 and total gold-answer log-probability retain the useful quality signal.

Dataset	Condition	n	Token F1	Gold log-prob.	Resident MiB
qasper	Ordinary split K/V	20	0.136	-28.66	0.0
qasper	Native FP K/V	20	0.136	-28.66	37.6
qasper	Native int8 resident	20	0.147	-27.73	18.8
qasper	Shuffled native K/V	20	0.057	-52.18	37.6
qasper	No memory	20	0.042	-54.53	0.0
hotpotqa	Ordinary split K/V	20	0.129	-10.95	0.0
hotpotqa	Native FP K/V	20	0.129	-10.95	42.0
hotpotqa	Native int8 resident	20	0.140	-10.21	21.0
hotpotqa	Shuffled native K/V	20	0.045	-19.48	42.0
hotpotqa	No memory	20	0.058	-15.68	0.0
2wikimultihopqa	Ordinary split K/V	20	0.059	-10.30	0.0
2wikimultihopqa	Native FP K/V	20	0.059	-10.30	42.0
2wikimultihopqa	Native int8 resident	20	0.059	-9.92	21.0
2wikimultihopqa	Shuffled native K/V	20	0.060	-16.39	42.0
2wikimultihopqa	No memory	20	0.073	-13.88	0.0

Table 8 shows exact full-precision transport parity: ordinary split-cache and native K/V have identical aggregate token F1 and gold answer log-probability on every completed dataset. On QASPER the two paths both reach 0.136 F1 and -28.66 mean log-probability; on HotpotQA both reach 0.129 and -10.95; on 2Wiki both reach 0.059 and -10.30. Shuffling memory lowers F1 to 0.057 and 0.045 on QASPER and HotpotQA and makes gold log-probability substantially worse on all three datasets. On 2Wiki, shuffled and no-memory F1 are slightly higher despite worse gold log-probability, exposing verbose generation as a noisy metric rather than supporting a quality gain. The exact transport parity and answer-likelihood controls establish source-dependent computation; broader decoding evaluation remains necessary.

Per-head int8 residency reduces mean selected storage from 37.6 to 18.8 MiB on QASPER and from 42.0 to 21.0 MiB on HotpotQA. Its aggregate F1 and gold log-probability are slightly higher in this small cohort, but this is not evidence that quantization improves quality: generation is discrete and the sample is small. The supported conclusion is non-degradation here plus a near-exact $2\times$ residency reduction. Dequantization restores model-native K/V before attention, so active materialization bytes are not reduced.

11.1 M4 Pro cross-model replication

The M4 Pro replication repeats all five execution conditions for 20 examples per dataset and model. It is independent of the earlier M5 run and adds Qwen3-4B. Table 9 reports the selector-frozen oracle-evidence comparison; Figure 3 keeps transport quality separate from execution cost.

Table 9: Cross-model original-answer QA on the M4 Pro. E0 is ordinary split-cache execution; E2 is the same selected evidence as native K/V. E2/E0 is completion latency, so lower is faster. FP/int8 is retained selected-K/V residency; int8 is dequantized before attention.

Model	Dataset	n	E0 F1	E2 F1	Shuf. F1	E2/E0	FP/int8 MiB
Qwen3-1.7B	2Wiki	20	0.070	0.070	0.059	0.529	42.0/21.0
Qwen3-1.7B	HotpotQA	20	0.143	0.143	0.038	0.535	42.0/21.0
Qwen3-1.7B	QASPER	20	0.153	0.153	0.031	0.545	37.6/18.8
Qwen3-4B	2Wiki	20	0.097	0.097	0.011	0.445	54.0/27.0
Qwen3-4B	HotpotQA	20	0.212	0.212	0.028	0.474	54.0/27.0
Qwen3-4B	QASPER	20	0.249	0.249	0.019	0.476	48.3/24.2

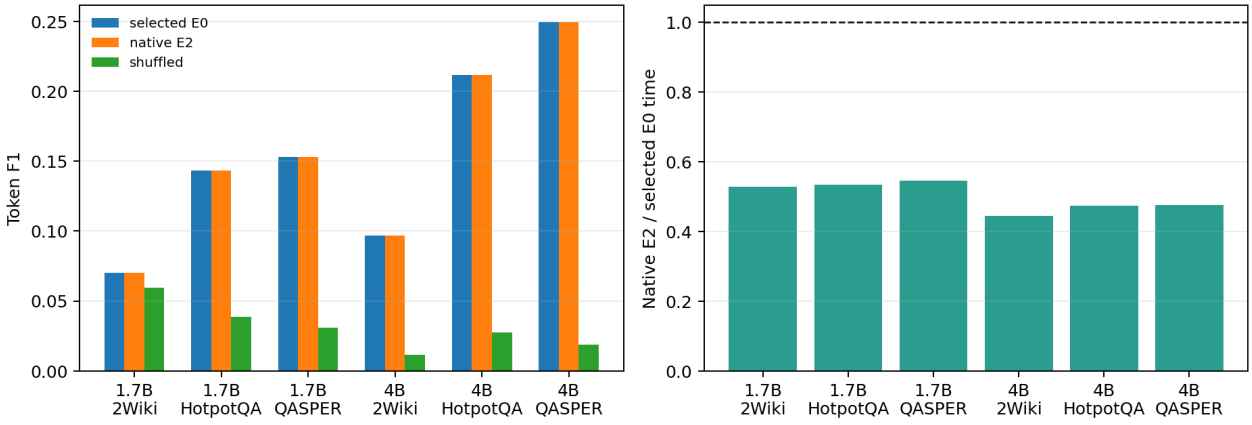

Figure 3: Cross-model transport and causal controls. Full-precision native E2 matches ordinary split-cache aggregate F1 in every cohort, while shuffled memory degrades quality. The latency panel measures the current MLX runner, not HTTP queueing or continuous batching.

At 1.7B, native E2 takes 52.9–54.5% of ordinary split-cache completion time; at 4B it takes 44.5–47.6%. Aggregate E0/E2 F1 is identical in all six cohorts. The 4B model improves E0/E2 F1 from 0.070 to 0.097 on 2Wiki, 0.143 to 0.212 on HotpotQA, and 0.153 to 0.249 on QASPER. Shuffled 4B memory falls to 0.011, 0.028, and 0.019, respectively, while its gold-answer log-probability drops by 6.81–38.58 nats relative to native E2. These interventions support a source-dependent native computation rather than query-only answer bias.

Full-precision residency is 37.6–42.0 MiB at 1.7B and 48.3–54.0 MiB at 4B; compact int8 residency halves each value. This is a retained-storage result: request-local dequantization still restores full-precision active K/V. The speed advantage over ordinary split-cache is specific to this in-process oracle-evidence runner and does not supersede the selector-frozen online E0/E2 benchmark, whose unfused native path remains slower under gateway schedules.

11.2 Routed evidence rather than oracle evidence

We next remove the oracle ordering. Each HotpotQA and 2Wiki article, or each QASPER paragraph, becomes a typed `AgentResource`. The product SDK's `PersistentResourceIndex` combines normalized token overlap, indexed BM25, and a deterministic signed-hash semantic control. It ranks the original question against all candidates; the top four are joined in rank order, truncated to the same 384-token budget, and encoded as native K/V. This is a parameter-free product-path baseline, not a learned retriever. Each dataset contains 20 distinct examples sampled under five seeds.

Table 10 exposes the discovery boundary. HotpotQA and 2Wiki have ten candidate documents per question and reach 0.725 and 0.775 recall@4. QASPER averages 46.6 paragraph candidates and reaches only 0.348. The

Table 10: Document routing before MLX materialization. Recall is the fraction of annotated evidence documents recovered in the first k ranks. Route time excludes one-time index construction.

Dataset	n	Candidates	R@1	R@2	R@4	Tokens	Route ms
qasper	20	46.6	0.119	0.140	0.348	370.6	11.92
hotpotqa	20	10.0	0.400	0.600	0.725	376.3	3.57
2wikimultihopqa	20	10.0	0.487	0.662	0.775	341.5	2.85

Table 11: Original-answer generation after routed materialization. Oracle uses annotation-scoped evidence; shuffled supplies another example's routed K/V.

Dataset	Condition	n	Token F1	Gold log-prob.
qasper	Oracle	20	0.148	-27.25
qasper	Routed	20	0.116	-38.36
qasper	Shuffled	20	0.034	-54.13
qasper	No memory	20	0.026	-60.20
hotpotqa	Oracle	20	0.129	-10.95
hotpotqa	Routed	20	0.091	-15.98
hotpotqa	Shuffled	20	0.054	-20.30
hotpotqa	No memory	20	0.058	-15.68
2wikimultihopqa	Oracle	20	0.059	-10.30
2wikimultihopqa	Routed	20	0.082	-14.31
2wikimultihopqa	Shuffled	20	0.054	-17.00
2wikimultihopqa	No memory	20	0.073	-13.88

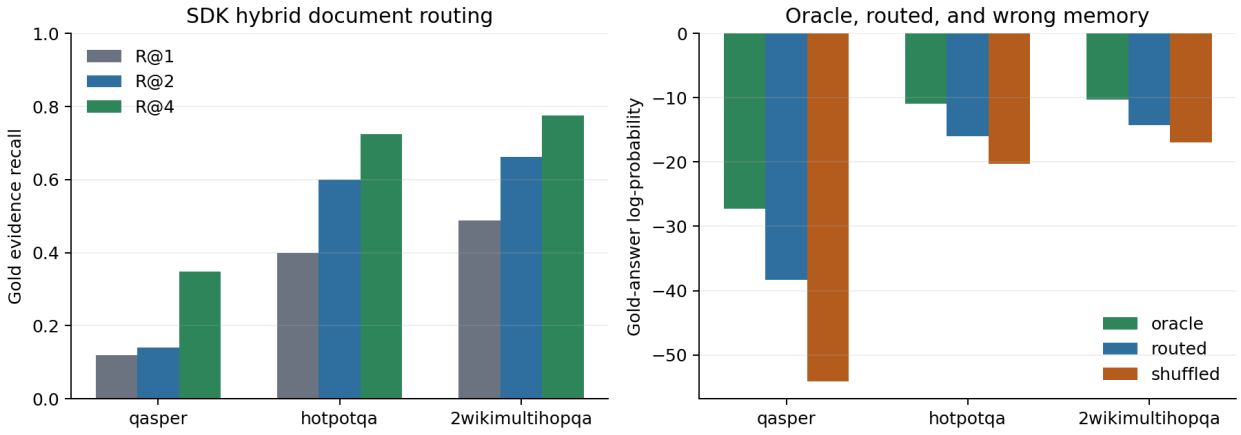

Figure 4: Left: evidence discovery improves as four candidates are admitted, but QASPER paragraph routing remains difficult. Right: routed memory carries more answer signal than shuffled memory, while the oracle gap remains.

selected sources consume 342–376 tokens. Mean scoring time is 2.85 ms on 2Wiki, 3.57 ms on HotpotQA, and 11.92 ms on QASPER; corresponding one-time index construction means are 6.56, 8.20, and 32.98 ms. These small exhaustive Python cohorts establish cost accounting, not a production latency claim.

The routed ordinary-cache and native-K/V paths produce the same output and exactly equal gold-answer log-probability on all 60 pairs. MLX consumption therefore adds no measured quality error after selection. Routed memory is better than shuffled memory in both F1 and gold likelihood on every dataset. It remains below oracle likelihood, however: QASPER changes from −27.25 to −38.36, HotpotQA from −10.95 to −15.98, and 2Wiki from −10.30 to −14.31. HotpotQA and 2Wiki routed likelihood also does not consistently beat the absent-memory control, despite better routed F1. Partial retrieval can be insufficient or misleading, while short free generation is noisy. The result closes the routed engine path and localizes the remaining deficit to evidence selection; it does not close a production QA-quality gate.

We then expand each dataset to 50 sampled examples (49 unique for QASPER and 50 for the other two). Table 12 confirms exact ordinary/native consumption parity and sharpens the dataset-specific result. On QASPER, routed native memory raises F1 from 0.038 without memory to 0.113, while shuffled memory reaches 0.042; on HotpotQA the corresponding values are 0.054, 0.113, and 0.034. Their gold-answer likelihoods move in the same causal direction. On 2Wiki, however, routed F1 is 0.076 versus 0.081 without memory and 0.089 shuffled, although routed likelihood remains better than shuffled. The larger cohort therefore confirms QASPER/Hotpot utility and a 2Wiki selection or answer-metric failure; it does not support a dataset-averaged quality claim.

Table 12: Expanded routed QA. Selection is frozen before ordinary or native consumption; oracle and shuffled conditions are causal bounds.

Dataset	Condition	Sampled/unique	Recall@4	F1	$\log p(y)$	Route ms
QASPER	no_memory	50/49	0.343	0.038	-51.37	14.27
QASPER	oracle_native	50/49	0.343	0.162	-22.83	14.27
QASPER	routed_native	50/49	0.343	0.113	-36.13	14.27
QASPER	routed_ordinary	50/49	0.343	0.113	-36.13	14.27
QASPER	routed_shuffled	50/49	0.343	0.042	-46.76	14.27
HotpotQA	no_memory	50/50	0.750	0.054	-17.99	4.17
HotpotQA	oracle_native	50/50	0.750	0.134	-11.09	4.17
HotpotQA	routed_native	50/50	0.750	0.113	-14.30	4.17
HotpotQA	routed_ordinary	50/50	0.750	0.113	-14.30	4.17
HotpotQA	routed_shuffled	50/50	0.750	0.034	-20.82	4.17
2Wiki	no_memory	50/50	0.725	0.081	-12.93	3.42
2Wiki	oracle_native	50/50	0.725	0.084	-11.53	3.42
2Wiki	routed_native	50/50	0.725	0.076	-14.12	3.42
2Wiki	routed_ordinary	50/50	0.725	0.076	-14.12	3.42
2Wiki	routed_shuffled	50/50	0.725	0.089	-15.22	3.42

11.3 Matched selected-text economics

The routed selections also support a direct E0/E2 comparison without changing the evidence. For 20 examples per dataset across five seeds, E0 restores an ordinary cache snapshot of the selected source and E2 restores its native PRA cache. The question, greedy decoder, and 24-token output budget are identical. The shared schedule includes cold one-shot, two warm repeats, three deterministic query formulations over the same resource, and eight threaded requests sharing one immutable cache object.

MLX-LM produces exactly the same E0 and E2 output in all 840 pairs, with equal F1 and gold evidence by construction. E2 reduces visible prompt tokens by 90.6–93.0%. Its mean time to usable context is 51.2 ms, but cold end-to-end cost is 1.130 times E0 because the unfused selected cache slows generation. Warm and multi-query ratios are 1.155 and 1.148. Under eight threaded requests, E2 reaches 88.4% of E0 throughput, an inverse cost ratio of 1.132. This is exact semantic transport across all four regimes, but not yet an optimized native-attention kernel.

The expanded 149-question confirmation preserves all 2,086 MLX pairs, with an engine-level Wilson 95% lower bound of 0.9982. Visible-token reduction remains 90.4–93.0%. Macro cold, warm, multi-query, and concurrent cost ratios are 1.043, 1.139, 1.133, and 1.072. The larger cohort thus reduces the measured cold and concurrent penalty relative to the original 20-example-per-dataset run, but confirms that this unfused implementation remains slower

Table 13: Matched-selection quality on the shared three-engine cohort. All parity pools 840 request pairs per engine across four schedules.

Engine	Dataset	Visible E0/E2	Reduction	Cold F1 E0/E2	Cold parity	All parity
mlx-lm	2wikimultihopqa	375/33	91.1%	0.067/0.067	100.0%	100.0%
mlx-lm	hotpotqa	415/39	90.6%	0.087/0.087	100.0%	100.0%
mlx-lm	qasper	399/28	93.0%	0.110/0.110	100.0%	100.0%
sglang-mlx	2wikimultihopqa	375/33	91.1%	0.074/0.074	100.0%	100.0%
sglang-mlx	hotpotqa	415/39	90.6%	0.084/0.084	100.0%	100.0%
sglang-mlx	qasper	399/28	93.0%	0.105/0.105	100.0%	100.0%
vllm-metal	2wikimultihopqa	378/33	91.2%	0.079/0.079	100.0%	100.0%
vllm-metal	hotpotqa	417/39	90.6%	0.074/0.074	100.0%	100.0%
vllm-metal	qasper	400/28	93.0%	0.101/0.101	100.0%	100.0%

Table 14: Macro E2/E0 execution-cost ratios; lower is better. Cold includes ingestion, concurrent is inverse requests/s, and E2 usable is milliseconds.

Engine	Cold	Warm	Multi-query	Concurrent	E2 usable ms
mlx-lm	1.130	1.155	1.148	1.132	51.2
sglang-mlx	0.976	1.136	1.140	1.122	55.0
vllm-metal	0.985	1.004	0.993	0.981	86.0

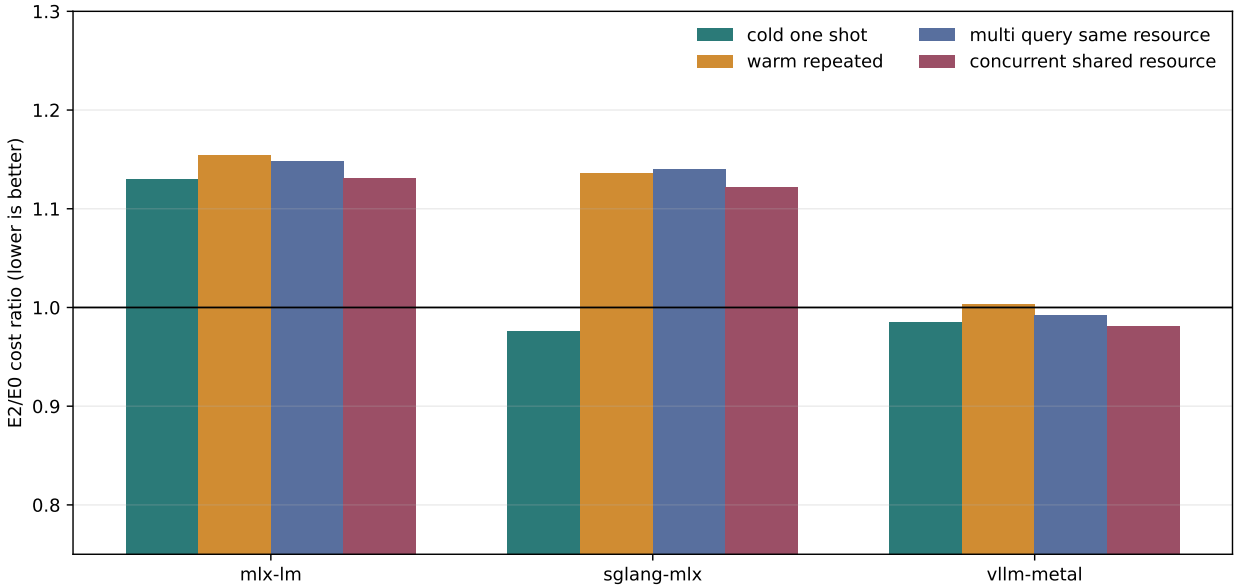

Figure 5: Macro representation cost after freezing retrieval. Lower is better; concurrent cost is inverse throughput. The current unfused MLX native cache is semantically exact but slower in every schedule.

than E0 in every regime.

We isolate the principal decode allocation with a one-normalization segmented attention primitive. Rather than concatenate selected and local K/V, it forms their score blocks separately, shares one maximum and denominator, and sums the two value numerators. At local windows 2K, 8K, and 32K with 384 selected tokens, eight heads, and 128-dimensional heads, maximum fp16 error against concatenated attention is 1.22×10^{-4}, 6.10×10^{-5}, and 6.10×10^{-5}. The primitive avoids 9.5, 33.5, and 129.5 MiB of K/V concatenation per layer-call. Mean M5 time changes from 0.754 to 0.684 ms, 2.515 to 1.597 ms, and 3.356 to 1.556 ms. These are isolated attention microbenchmarks, not revised end-to-end E2 ratios: installing the primitive in each model's GQA/MQA attention path remains an engine patch.

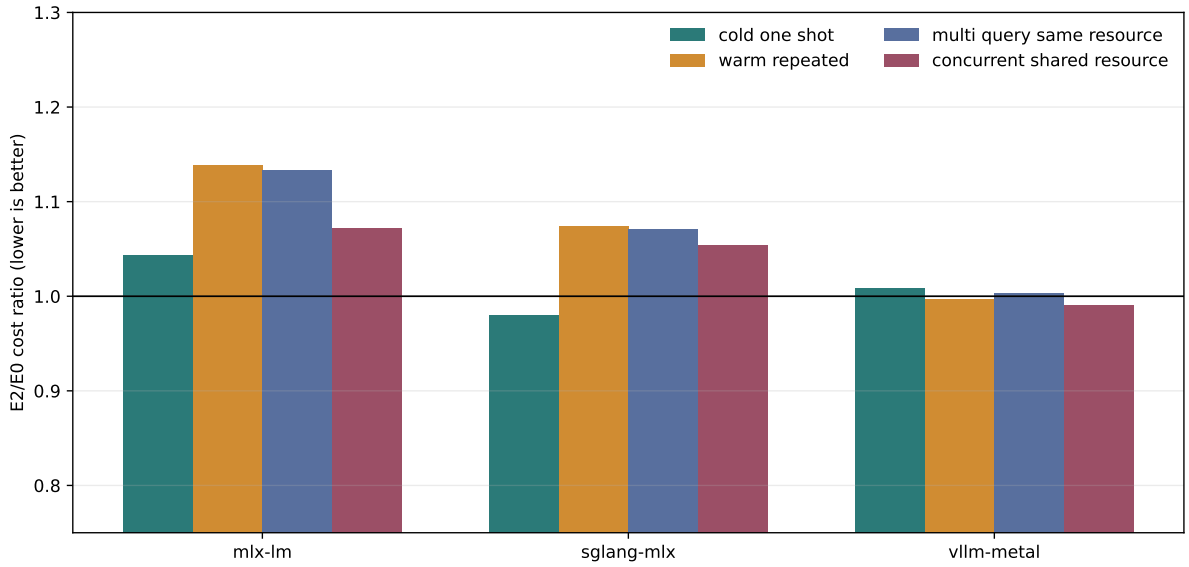

Figure 6: Expanded 149-question representation-cost confirmation across all three engines. Retrieval and selected evidence are held fixed.

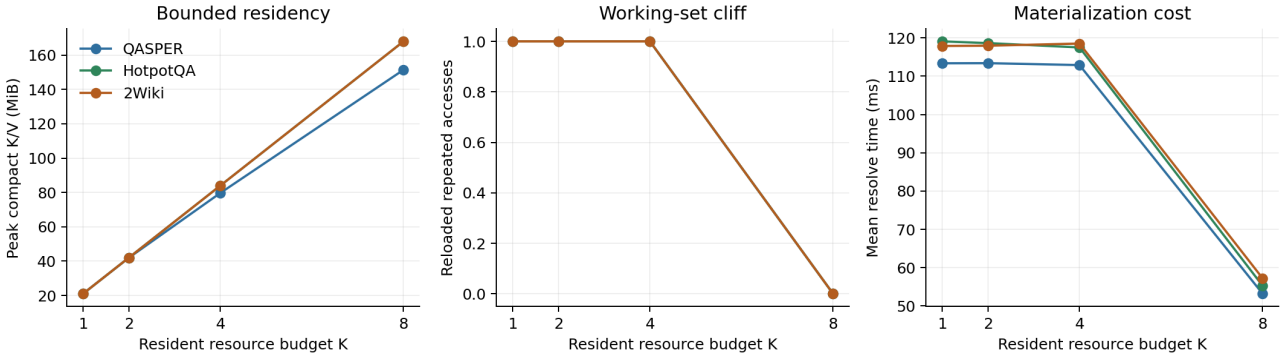

Figure 7: Five-seed compact-K/V pressure over QASPER, HotpotQA, and 2Wiki. Each seed visits eight resources twice and then revisits the first. The budget $K = 8$ is the first condition that fits the working set.

Figure 7 and Table 15 validate physical lifecycle rather than theoretical byte sums. Each request pins one selected block, dequantizes it locally, generates, and releases it. With budgets $K \in \{1, 2, 4\}$, the cyclic eight-resource working set reloads all nine repeated accesses per seed. At $K = 8$, initial loads fall from 17 to eight, evictions and reloads fall to zero, and mean resolution falls from 113–119 ms to 53–57 ms. Peak compact residency rises from approximately 21 MiB at $K = 1$ to 151 MiB on QASPER and 168 MiB on HotpotQA and 2Wiki at $K = 8$.

Token F1 is unchanged across all four budgets within every dataset, and mean completion latency remains within roughly 5 ms because generation dominates the saved resolution time. The result is therefore a working-set

Table 15: Natural-QA residency curve. Each row contains 85 requests: 17 requests per seed over five seeds. Reload fraction divides reloads by the nine accesses after each resource's first load.

Dataset	K	n	Peak MiB	Reload frac.	Resolve ms	Token F1
QASPER	1	85	21.0	1.00	113.4	0.189
QASPER	2	85	41.9	1.00	113.4	0.189
QASPER	4	85	79.7	1.00	112.9	0.189
QASPER	8	85	151.4	0.00	53.2	0.189
HotpotQA	1	85	21.0	1.00	119.1	0.144
HotpotQA	2	85	42.0	1.00	118.6	0.144
HotpotQA	4	85	84.0	1.00	117.5	0.144
HotpotQA	8	85	168.0	0.00	55.2	0.144
2Wiki	1	85	21.0	1.00	117.9	0.049
2Wiki	2	85	42.0	1.00	117.9	0.049
2Wiki	4	85	84.0	1.00	118.5	0.049
2Wiki	8	85	168.0	0.00	57.2	0.049

threshold, not evidence that more residency generally improves answer quality or complete request latency. No pinned block is evicted and compact detail never enters an ordinary prompt or rotating cache.

11.4 Larger-model sustained pressure on M4 Pro

We repeat the bounded-residency experiment with Qwen3-4B on the M4 Pro. Each seed owns eight resources, visits the complete set three times, and revisits the first resource. Four resident budgets produce 25 requests per seed, or 125 requests per table row and 1,500 requests overall.

Table 16: Qwen3-4B sustained compact-K/V pressure on the M4 Pro. Budget is resident/logical resources. Resolve includes compact-detail load and dequantization; p95 retains unavoidable first-ingestion requests.

Dataset	Budget	n	F1	Reload	Resolve ms	p95 ms	Resident MiB
QASPER	1/8	125	0.218	0.680	417.1	460.6	24.4
QASPER	2/8	125	0.218	0.680	420.0	460.7	47.7
QASPER	4/8	125	0.218	0.680	419.3	460.5	91.5
QASPER	8/8	125	0.218	0.000	134.3	460.2	167.4
HotpotQA	1/8	125	0.231	0.680	457.2	460.2	27.0
HotpotQA	2/8	125	0.231	0.680	457.8	461.0	52.9
HotpotQA	4/8	125	0.231	0.680	457.6	461.1	101.5
HotpotQA	8/8	125	0.231	0.000	146.5	460.1	185.8
2Wiki	1/8	125	0.128	0.680	458.8	467.1	27.0
2Wiki	2/8	125	0.128	0.680	461.1	478.1	52.9
2Wiki	4/8	125	0.128	0.680	462.2	480.6	101.5
2Wiki	8/8	125	0.128	0.000	147.1	465.8	185.8

For budgets 1, 2, and 4, 68.0% of all requests reload compact detail. At budget 8, reloads fall to zero. Mean resolution falls from 417.1 to 134.3 ms on QASPER, 457.2 to 146.5 ms on HotpotQA, and 458.8 to 147.1 ms on 2Wiki, a 67.8–68.0% reduction. Mean compact residency rises to 167.4 MiB on QASPER and 185.8 MiB on HotpotQA and 2Wiki. Token F1 and gold-answer log-probability are identical across all four budgets within each dataset.

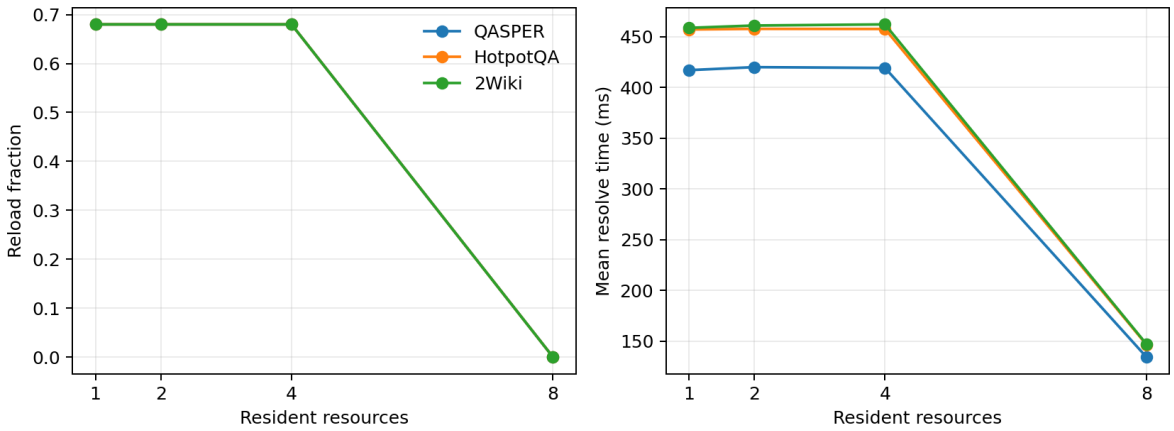

Figure 8: The eight-resource working-set threshold at Qwen3-4B. Fitting the complete set removes repeat reloads and lowers mean resolution; it does not remove each resource's first cold ingestion.

The p95 resolve time remains 460–466 ms at budget 8 because eight of each 25-request sequence are compulsory first loads. Residency removes 17 repeat loads, not those cold ingestions. This result therefore supports working-set admission and reuse policy; it does not support a claim that larger budgets improve answer quality or cold-start tails.

11.5 Shared storage lifecycle and MLX realization

The product runtime now defines engine-neutral HOT/WARM/COLD/SOURCE semantics. MLX attention-ready arrays are HOT. Lossless retained arrays or strict fingerprinted files are WARM. Compressed persistence is COLD, with K/V quantization configured independently from compression. Apple unified memory can physically host both HOT and WARM, but readiness and lifecycle remain different service states.

Across the common four-label, five-seed policy workload, lossless tier recovery and SOURCE reconstruction are exact in 20/20 runs. Weighted retention keeps the shared document where plain LRU does not; active-task and dependency guards hold in 20/20, and a grace period avoids 20/20 transient writes. A generic per-array symmetric-int8 codec occupies 0.25 of FP32 bytes with 0.0103 RMSE. That codec result is not an MLX answer-quality claim. The model-native per-head experiment in this paper is the applicable quality evidence and obtains a roughly twofold compact-residency reduction because its metadata and scale layout differ. COLD still dequantizes before attention, so compact persistence does not reduce active attention bytes.

The live storage probe now replaces opaque WARM blobs with independently checksummed, mmap-readable K and V segments per layer. Three QASPER selections generate from MLX HOT arrays, lossless WARM reload, int8 COLD reload, and a fresh manager after restart. WARM reproduces all 3/3 HOT outputs and restart recovery succeeds. Median promotion-plus-generation latency is 327 ms HOT and 644 ms WARM (1.97×); the excluded background WARM transition costs 348 ms. COLD promotion plus generation is 600 ms and its background conversion costs 940 ms.

Table 17: Shared three-example live storage probe. Prefix is mean HOT/int8 common output prefix; ratios use lifecycle-inclusive request latency.

Engine	WARM exact	int8 exact	int8 first	Prefix	WARM/HOT	COLD/HOT
mlx-lm	100%	0%	67%	1.3	1.97	1.84
sglang-mlx	100%	0%	67%	0.7	1.90	1.89
vllm-metal	100%	0%	67%	4.0	1.96	2.00

Per-array int8 COLD changes all three 24-token sequences and preserves the first token in two. Its positive mean F1 change (+0.068) is an unstable three-example accident rather than evidence of improved quality: exact parity is zero and the mean common prefix is 1.3 tokens. Lossless persistence is the default; quantized COLD

requires a larger model- and workload-specific quality gate. The separate per-head compact-residency experiment remains relevant but does not override this autoregressive sequence diagnostic.

We expand the protocol to 80 examples: all three datasets at Qwen3-0.6B and QASPER at Qwen3-1.7B. Lossless WARM remains 80/80 exact and all four managers recover after restart; int8 COLD is exact in 13/80 sequences. WARM/HOT p50 ratios are 2.01–2.15, with WARM p95 between 710 and 817 ms. The larger cohort therefore strengthens persistence correctness while making request-time promotion, rather than array storage, the immediate systems bottleneck. Two five-example QASPER replications add Llama-3.2-1B and Gemma-3-1B; both are 5/5 lossless-WARM exact with restart recovery. Llama int8 is 2/5 exact and Gemma is 5/5 in this small cohort, which is insufficient to override the broader Qwen-based opt-in gate.

Table 18: Expanded MLX lifecycle cohorts. Latencies include synchronous promotion and generation; int8 remains diagnostic.

Engine	Model	Data	n	WARM exact	int8 exact	HOT p50	WARM p50	WARM p95
mlx-lm	Llama-3.2-1B-Instruct-4bit	qasper	5	100.0%	40.0%	191	275	281
mlx-lm	Qwen3-0.6B-4bit	2wikimultihopqa	20	100.0%	15.0%	328	658	710
mlx-lm	Qwen3-0.6B-4bit	hotpotqa	20	100.0%	15.0%	329	707	817
mlx-lm	Qwen3-0.6B-4bit	qasper	20	100.0%	15.0%	326	672	714
mlx-lm	Qwen3-1.7B-4bit	qasper	20	100.0%	20.0%	337	723	744
mlx-lm	gemma-3-1b-it-4bit	qasper	5	100.0%	100.0%	391	480	502

11.6 Asynchronous promotion and online request behavior

The lifecycle ratios above charge a complete WARM promotion to the requesting thread. We therefore move promotion ownership to the event loop, deduplicate concurrent promotions by logical key, admit completed objects to a bounded hot set, and overlap promotion with query-side work. Table 19 and Figure 9 report five QASPER selections at each lead. Every generation remains HOT-exact. With no lead, median demand stall is 383.7 ms and demand-path cost is 2.123 times HOT. A 250 ms lead lowers these to 90.3 ms and 1.310; 350 ms makes 40% ready at demand and reaches 1.048; 500 ms makes all five ready and reaches 0.983. Promotion is therefore hideable when the scheduler has enough useful work, but it has not become intrinsically cheaper.

Table 19: Event-loop-owned WARM promotion. Lead and stall are milliseconds.

Engine	Lead	Ready	Stall p50	Stall p95	Demand/HOT
mlx	0 ms	0%	383.7	397.6	2.123
mlx	25 ms	0%	332.7	358.1	2.054
mlx	50 ms	0%	304.3	328.3	1.962
mlx	100 ms	0%	256.4	305.5	1.836
mlx	250 ms	0%	90.3	122.4	1.310
mlx	350 ms	40%	4.2	40.8	1.048
mlx	500 ms	100%	0.1	0.1	0.983
sglang	0 ms	0%	210.3	280.1	1.561
sglang	25 ms	0%	202.4	240.7	1.495
sglang	50 ms	0%	169.0	195.7	1.475
sglang	100 ms	0%	111.9	141.4	1.326
sglang	250 ms	80%	0.0	5.7	1.035
sglang	350 ms	100%	0.0	0.1	1.063
sglang	500 ms	100%	0.0	0.0	1.023

The native executor is also exercised through the OpenAI-compatible gateway, including streaming, cancellation, cleanup, and session ownership. At concurrency one through eight, throughput is 1.56–1.66 requests/s; TTFT p95

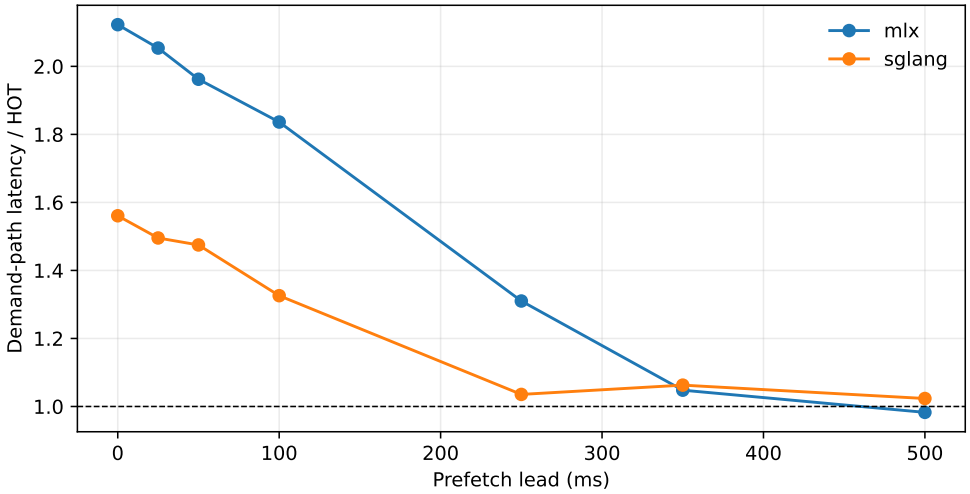

Figure 9: Promotion lead trades demand-path stall for speculative work. The dashed line is HOT request latency.

rises from 324 to 4,845 ms as the deliberately serialized model runner queues. The client cancels after the first streamed token (338 ms) and all 18 exercised sessions close without retaining request-owned native detail. This is a real HTTP-path measurement, but the single MLX model instance is intentionally serialized to avoid unsafe concurrent mutation of prompt-cache arrays.

Table 20: Online native-gateway concurrency, including queueing. Times are ms.

Engine	c	req/s	TTFT p50	TTFT p95	request p50	request p95
mlx-lm	1	1.65	324	324	607	607
mlx-lm	2	1.66	336	925	615	1202
mlx-lm	4	1.56	1006	2269	1291	2561
mlx-lm	8	1.56	2384	4845	2663	5126
sglang-mlx	1	3.20	40	40	311	311
sglang-mlx	2	3.17	34	360	332	631
sglang-mlx	4	3.12	357	988	655	1280
sglang-mlx	8	3.04	1051	2337	1356	2625

11.7 Concurrent physical-tier curves

The lifecycle manager is next stressed at concurrency $c \in \{1, 2, 4, 8, 16\}$ with either one shared immutable resource or one resource per request. The model runner remains deliberately serialized, so these finite waves measure queueing, promotion, and physical sharing rather than continuous batching. All 124 HOT/WARM requests reproduce their HOT baselines exactly. At $c = 16$, shared HOT and WARM sustain 3.20 and 2.91 requests/s with request p95 of 4,996 and 5,491 ms. The shared WARM wave promotes one object and avoids 630 MiB of duplicate physical K/V. Independent WARM instead promotes 16 objects, reads 1.26 GiB, reaches 1.22 requests/s, and has 13,087 ms p95. The gap isolates resource sharing and promotion amplification; it is not evidence for parallel model execution. Int8 COLD remains outside the lossless gate.

11.8 Selective quantization and sustained pressure

Twenty QASPER examples isolate which tensors can tolerate int8 COLD storage. Quantizing every key and value is sequence-exact in 4/20 cases. Keys alone are 5/20; values alone are 10/20; the late half is 11/20; and the late quarter is 14/20, with first-token parity in all 20 and a mean 18.85-token common prefix. Aggregate F1 changes

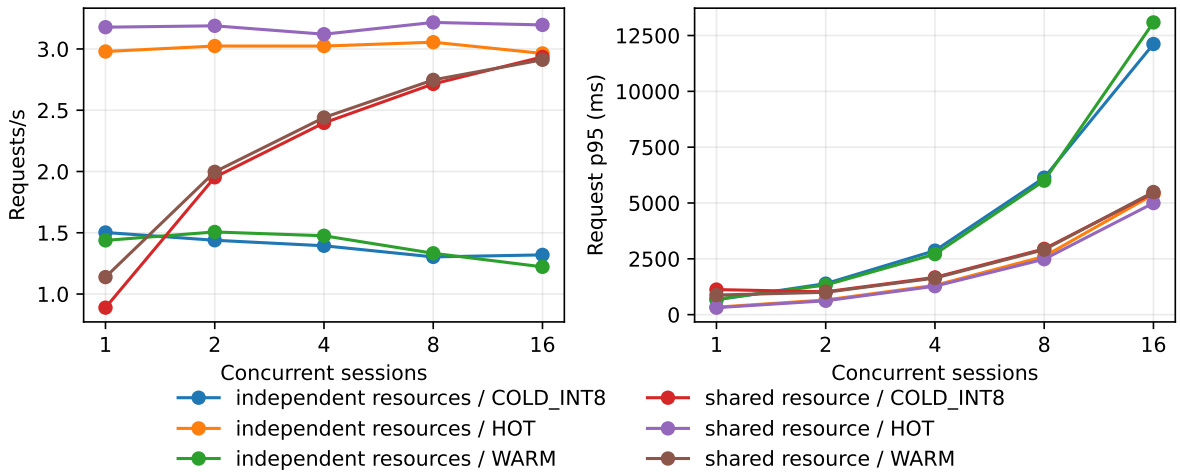

Figure 10: MLX physical-tier concurrency. Shared WARM amortizes one promotion; independent WARM scales storage work with the number of resources.

Table 21: Complete finite-wave storage-concurrency results. Times are ms; exact/HOT is the transport-fidelity gate.

Workload	Tier	c	req/s	p50	p95	p99	queue p95	exact/HOT
shared_resource	hot	1	3.18	313	313	313	0	100.0%
shared_resource	hot	2	3.19	314	621	621	308	100.0%
shared_resource	hot	4	3.12	626	1274	1274	944	100.0%
shared_resource	hot	8	3.22	1226	2484	2484	2165	100.0%
shared_resource	hot	16	3.20	2470	4996	4996	4688	100.0%
independent_resources	hot	1	2.98	335	335	335	0	100.0%
independent_resources	hot	2	3.02	331	657	657	328	100.0%
independent_resources	hot	4	3.02	666	1313	1313	997	100.0%
independent_resources	hot	8	3.06	1328	2607	2607	2274	100.0%
independent_resources	hot	16	2.96	2781	5384	5384	5082	100.0%
shared_resource	warm	1	1.14	877	877	877	0	100.0%
shared_resource	warm	2	2.00	710	1001	1001	710	100.0%
shared_resource	warm	4	2.44	999	1638	1638	1318	100.0%
shared_resource	warm	8	2.75	1648	2909	2909	2593	100.0%
shared_resource	warm	16	2.91	2950	5491	5491	5176	100.0%
independent_resources	warm	1	1.44	695	695	695	0	100.0%
independent_resources	warm	2	1.51	723	1327	1327	723	100.0%
independent_resources	warm	4	1.48	1377	2710	2710	1963	100.0%
independent_resources	warm	8	1.33	2938	5999	5999	5246	100.0%
independent_resources	warm	16	1.22	6673	13086	13086	12310	100.0%
shared_resource	cold_int8	1	0.89	1123	1123	1123	0	0.0%
shared_resource	cold_int8	2	1.95	708	1024	1024	708	0.0%
shared_resource	cold_int8	4	2.40	1037	1668	1668	1358	0.0%
shared_resource	cold_int8	8	2.71	1686	2945	2945	2640	0.0%
shared_resource	cold_int8	16	2.94	2901	5446	5446	5125	0.0%
independent_resources	cold_int8	1	1.50	665	665	665	0	0.0%
independent_resources	cold_int8	2	1.44	715	1389	1389	715	0.0%
independent_resources	cold_int8	4	1.39	1403	2867	2867	2092	0.0%
independent_resources	cold_int8	8	1.30	3173	6135	6135	5373	0.0%
independent_resources	cold_int8	16	1.32	6179	12119	12119	11329	12.5%

remain small, but autoregressive divergence makes even the late-quarter profile a calibration candidate rather than a default.

Table 22: Selective int8 COLD calibration on 20 QASPER examples.

Profile	Exact	First	Prefix	ΔF1	$\Delta \log p$
lossless	100.0%	100.0%	24.0	+0.0000	+0.0000
all_kv	20.0%	95.0%	8.8	-0.0163	+0.2862
all_keys	25.0%	85.0%	9.8	-0.0039	+0.0905
all_values	50.0%	100.0%	14.6	+0.0121	+0.1798
early_half_kv	15.0%	85.0%	8.6	+0.0012	+0.2602
late_half_kv	55.0%	95.0%	16.6	-0.0028	+0.0771
late_quarter_kv	70.0%	100.0%	18.9	-0.0031	+0.0698

A separate sustained-session study issues 1,125 requests over QASPER, HotpotQA, and 2Wiki: five seeds, eight resources, three complete rounds and a final revisit, under HOT capacities two, four, and eight. Capacities two and four average 17 reloads per seed and reload the final revisit in every case; capacity eight has no reload or eviction. F1 and answer log-probability are identical across budgets within each dataset. The intervention changes physical reuse, not selected evidence or model semantics.

Table 23: Long-session pressure. Reloads and evictions are means over five seeds.

Dataset	HOT resources	Requests	Reloads	Evictions	Final reload	F1	p95 ms
qasper	2	125	17.0	23.0	100%	0.172	493
qasper	4	125	17.0	21.0	100%	0.172	493
qasper	8	125	0.0	0.0	0%	0.172	495
hotpotqa	2	125	17.0	23.0	100%	0.131	497
hotpotqa	4	125	17.0	21.0	100%	0.131	500
hotpotqa	8	125	0.0	0.0	0%	0.131	501
2wikimultihopqa	2	125	17.0	23.0	100%	0.064	497
2wikimultihopqa	4	125	17.0	21.0	100%	0.064	508
2wikimultihopqa	8	125	0.0	0.0	0%	0.064	502

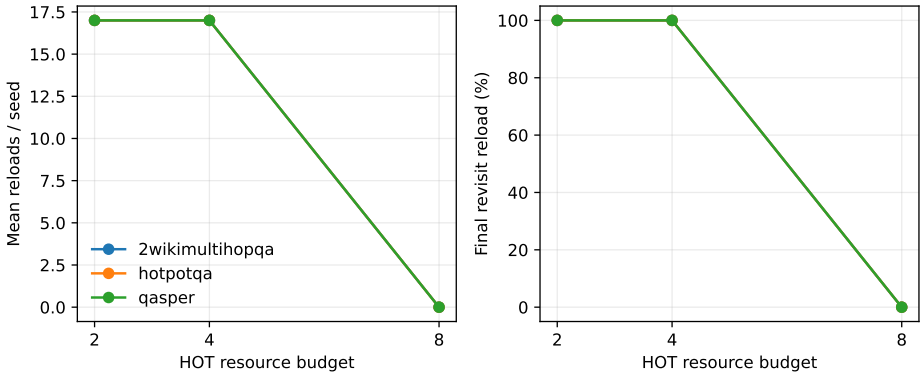

Figure 11: The eight-resource working-set threshold appears consistently across all three datasets.

The final pressure experiment crosses physical HOT and WARM budgets with local rotating windows of 64 and 256 tokens over 1,000 QASPER requests. With HOT/WARM = 2/2, 75.2% of requests reconstruct from SOURCE; raising WARM to eight removes SOURCE starts. With HOT/WARM = 8/2, starts are 68% HOT, 8% WARM,

and 24% SOURCE; at 8/8, the latter two become 32% WARM and zero SOURCE. F1 is 0.123 in every cell, and changing the local window does not materially move completion p95. Disk-backed WARM also fails to improve resolve p95 over this mixture of small-source reconstruction: retaining all eight objects reaches 400–429 ms, while the tighter WARM budget reaches 347–398 ms. The product implication is cost-aware admission, not maximal persistence.

The deterministic admission calculation makes that implication concrete. Across this pressure trace, SOURCE reconstruction averages 335.7 ms and WARM restore averages 346.0 ms; creating the WARM representation costs another 396.8 ms on average. Thus

$$r(C_{\text{source}} - C_{\text{warm}}) > C_{\text{persist}}$$

has no positive break-even reuse count on this cohort. The correct default for these small reconstructable QASPER records is SOURCE, not WARM. This is a hardware- and representation-specific decision, not a rejection of persistent K/V for expensive encoders or remote sources.

The requested 2K, 8K, 32K, and full local-history frontier is not inferred from the 64/256-token cache-capacity control. Those cells require prompts that actually occupy each window. We therefore label the current joint frontier partial: resource budgets 2/8, concurrency 1–16, and consumer-layer profiles are measured, while long local-window occupancy remains a separate staged run. The last-half consumer profile preserves 5/5 controlled recoveries at half the active selected K/V bytes; last-quarter and last-four do not, so neither is a BALANCED or ECONOMY recommendation.

Table 24: Crossed physical-tier and local-window pressure. Start percentages name the representation available before each request. Times are ms.

HOT	WARM	Window	Requests	HOT start	WARM start	SOURCE start	F1	Resolve p95	Total p95
2	2	64	125	0%	25%	75%	0.123	398.2	354
2	2	256	125	0%	25%	75%	0.123	377.9	353
2	8	64	125	0%	100%	0%	0.123	428.6	353
2	8	256	125	0%	100%	0%	0.123	428.4	354
8	2	64	125	68%	8%	24%	0.123	351.9	345
8	2	256	125	68%	8%	24%	0.123	347.2	348
8	8	64	125	68%	32%	0%	0.123	402.8	346
8	8	256	125	68%	32%	0%	0.123	400.0	349

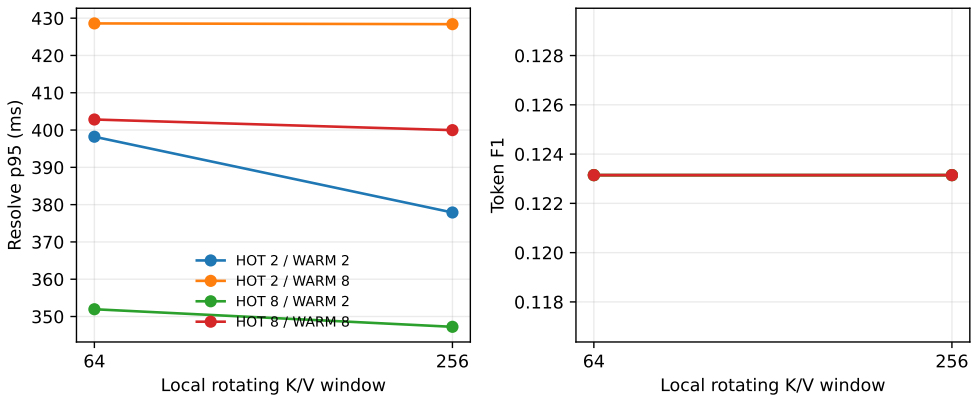

Figure 12: Local-window size has no quality effect in this bounded decode; HOT admission dominates median resolution, while broad WARM retention does not improve the tail on this host.

12 Promotion Gates

Gate	Status	Evidence
environment	closed	MLX Metal generation and server repeat reliably
prompt-cache coexistence	smoke closed	364/365 selected tokens reused warm
rotating-cache control	controlled closed	five seeds, four capacities
native semantic parity	controlled closed	five seeds, zero logit error, 5/5 recovery
rotating plus native	controlled closed	5/5 with 64-token local cache
lazy native region	prototype closed	selected resources encode on first selection
consumer profile	controlled candidate	last 14/28 layers retain 5/5 at half K/V bytes
persistence	controlled closed	strict fingerprint; zero-error reload in 7.2 ms mean
shared storage lifecycle	controlled closed	segmented WARM 90/90 exact across Qwen/Llama/Gemma; restart recovery; int8 remains gated
concurrency/sharing	controlled closed	five seeds at 1–8 requests; 108 MiB avoided at eight
natural transport	controlled closed	80/80 native; disabled/shuffled near chance
natural QA transport	controlled closed	original QASPER/Hotpot/2Wiki answers with causal controls
routed natural QA	controlled partial	60/60 consumption parity; recall@4 0.348–0.775; oracle gap open
matched E0/E2 economics	controlled closed	2,086/2,086 expanded outputs; all four E2/E0 cost ratios exceed one
quantized PRA detail	controlled candidate	per-head int8 halves residency; native dequantization before attention
memory pressure	controlled closed	3,145 requests across compact, long-session, and crossed tier/window pressure
SOURCE/WARM admission	controlled closed	WARM is slower than SOURCE on the measured small-record cohort; no positive break-even reuse
large local-window Pareto	open	2K/8K/32K/full require occupied long histories and are not inferred from cache limits

13 Falsification and Limitations

Native PRA-MLX is unnecessary if selected visible text plus ordinary MLX caching matches its quality, memory, and latency on realistic long-memory workloads. The server smoke makes that baseline competitive. Conversely, the rotating control shows that bounded sequential KV is not automatically a PRA archive.

The profile and original natural controls center on one 4-bit Qwen model and one machine; Llama/Gemma lifecycle rows are smaller mechanism replications. The initial server experiment has only five repeats per condition, so its p95/p99 are not reported; the later gateway load curve reports finite-wave tails but serializes one model runner. The natural controls contain real dataset text but an inserted fixed answer-code mechanism. The first end-task study uses original answers but oracle-scoped evidence. The richer M4 replication still uses only two Qwen sizes and 20 examples per dataset/model; it establishes cross-scale transport and causal sensitivity, not broad model-family generalization. The routed extension now has 50 sampled examples per condition and dataset (49 unique on QASPER), a parameter-free lexical/hash index, and four selected candidates. Strict EM is confounded by verbose continuation. Int8 detail is restored before attention, so it saves retained bytes rather than active attention bytes. No learned native Q/K index, continuous batching, or OOM frontier is measured. The residency sweeps use deterministic cyclic schedules rather than a measured production session distribution. The original matched benchmark reuses only 20 examples per dataset; the expanded selector-frozen confirmation increases that cohort but does not establish end-task quality beyond proving that E2 preserves its E0 input. The rotating intervention also changes effective local context and can perturb chat-template semantics.

14 Related Work

MLX targets efficient and flexible machine learning on Apple Silicon [1]. Prompt caching and rotating K/V optimize sequential state; cache quantization reduces its physical size. Retrieval and context compression reduce visible

input. PRA instead assigns stable identity to retained non-prefix memory and separates cheap addressing from active native detail. The mechanisms can be combined, but none is a semantic substitute for the others.

15 Conclusion

MLX prompt caching and selected context compose cleanly: the fixed answer is preserved with 76.9% fewer visible tokens, and repeated selected prompts reuse 364 of 365 tokens. Rotating K/V provides large layer-cache savings but fails as an archive in the five-seed control; late selected text does not reliably repair it. The in-process native executor passes the missing mechanism gate: selected non-prefix K/V is bit-exact to ordinary split-cache execution and preserves 5/5 recovery with a rotating local cache. PRA-MLX is therefore a working native prototype. Full-precision native transport also matches ordinary split-cache quality on original-answer QA, while wrong and absent memory fail causally. Per-head int8 halves retained detail without measured degradation in the small cohort. The M4 Pro replication extends this result to Qwen3-4B: ordinary and native aggregate F1 remain equal in all six cohorts, shuffled memory degrades sharply, and native completion uses 44.5–54.5% of ordinary split-cache time in this in-process runner. Its 1,500-request pressure extension reproduces the eight-resource residency threshold and separates repeat-load savings from cold p95. Across a 1,020-request hard-budget sweep, sub-working-set budgets cap compact residency but reload every repeated access; fitting all eight resources removes reloads and halves resolution time without changing F1. With SDK routing, native and ordinary paths remain identical on 60/60 paired generations, but recall@4 ranges from 0.348 to 0.775 and answer likelihood remains below oracle evidence. The next quality improvement belongs in discovery, not the MLX cache transport. Queued server pressure, an OOM frontier, learned routing, and broader quality cohorts remain necessary before production claims. Matched E2 execution removes roughly 91–93% of visible tokens and preserves every output, but its unfused decode is slower. Last-half consumption and int8 residency are controlled candidates, not product profile defaults. A segmented one-softmax primitive removes the dominant K/V concatenation in isolation, but the model runner is not called fused until that primitive replaces the live attention path.

References

[1] A. Hannun et al. MLX: Efficient and Flexible Machine Learning on Apple Silicon. 2023.